

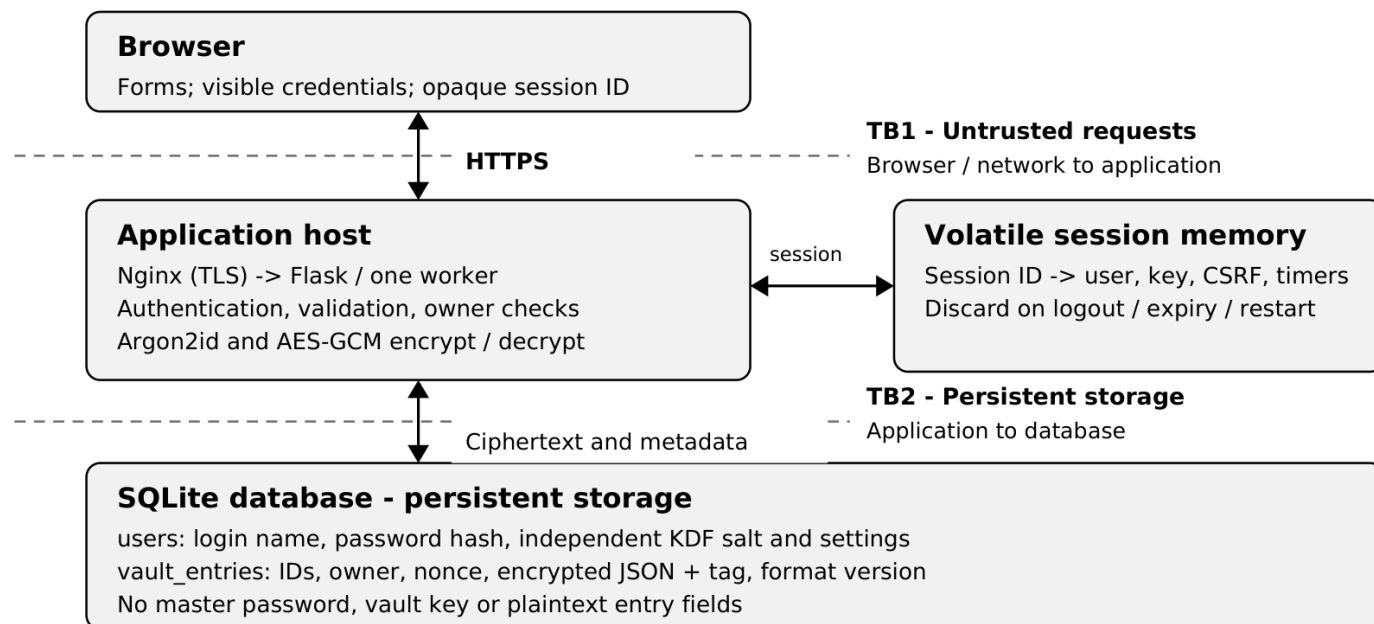
Web Based Secure Password Manager

Checkpoint 1 Threat model and architecture

ICS0027 | 21 September 2026 | Proposed design | Danila Sisov

1 Scope and architecture

The application will let registered users add, view, edit and delete their own website credentials. An entry contains a title, website URL, username, password and optional notes. The master password is used to sign in and unlock the vault. The initial scope excludes sharing, file uploads, browser extensions, import/export, password recovery and an admin interface. Optional MFA is deferred.



TB1 - Browser to server. Requests cross an untrusted network and use HTTPS. The browser is not a security authority: the server validates input, the session, CSRF tokens and ownership. The browser sees credentials when the user types or views them. Vault encryption and decryption happen in the Flask application.

TB2 - Application to database. The database is persistent storage, not a key store. The application writes encrypted entries and checks their authentication tags when reading. Database access is local and restricted to the application OS account. It is not exposed through a public route.

Component	Choice and reason
Web application	Python 3.11+, Flask and Jinja: small server-rendered forms; little JavaScript.
Database	SQLite through sqlite3: simple local storage and parameterized queries.
Cryptography	argon2-cffi for password verification/key derivation; cryptography AESGCM for authenticated encryption.
HTTPS	Planned Nginx TLS endpoint and one Gunicorn worker on the same host; details in section 3.

Assumptions and assets. Protect master passwords, vault contents, encryption keys and session identifiers. Consider anonymous attackers, malicious account holders, network observers and someone with a stolen database copy. The server software and OS are trusted. A compromised browser or running server can capture unlocked secrets; this design does not prevent that. These are educational scope choices, subject to any separate final-project requirements.

2 Cryptographic design

Master password and separate outputs

At registration, the server will produce two independent outputs from the master password. It will not store the master password.

Purpose	Planned operation	Persistent value
Verify login	Argon2id using PasswordHasher.hash() with its own random salt	Encoded password hash, including authentication salt and parameters
Encrypt vault entries	Argon2id using hash_secret_raw() with a separate random 16-byte kdf_salt	Salt and KDF parameters only; never the derived key

Use Argon2id version 19, 65,536 KiB memory, 3 iterations, parallelism 4 and a 32-byte output for both operations. These are explicit initial settings based on the library's documented low-memory profile. Verify their cost on the development machine before implementing login. The independent salts ensure that the stored login-verification value is not the vault key.

On login, PasswordHasher.verify() checks the supplied password. Only after successful verification will the server derive the vault key again using that user's stored KDF salt and settings. The raw 32-byte result is the AES-256 key. It is held only in the server's active session memory. The master password is not kept in the session. No key is written to a cookie, browser storage, log or database.

What is encrypted

For each entry, serialize its title, website URL, username, password and notes as UTF-8 JSON. Encrypt the entire JSON value on the server with AES-256-GCM. Generate a fresh random 12-byte nonce for every encryption, including edits; never deliberately reuse a nonce with the same key. Store the ciphertext and its 16-byte authentication tag, which AESGCM.encrypt() returns together.

Bind the ciphertext to its owner and entry using authenticated additional data (AAD): the UTF-8 string user_id:entry_id:format_version, using server-assigned integer IDs. Allocate the entry ID inside the database transaction before encryption. The same AAD is required to decrypt. A failed tag check returns a generic error and no plaintext; it does not fall back to unencrypted data.

Planned database contents

Table	Stored fields
users	id, unique login_username, encoded password_hash, kdf_salt, kdf_memory, kdf_iterations, kdf_parallelism, kdf_version
vault_entries	id, user_id (foreign key), nonce, encrypted_data (ciphertext plus tag), format_version

Login names, IDs, owner relationships, salts, KDF settings and record sizes remain visible in a database copy. Website names, URLs, saved usernames, passwords and notes are inside the ciphertext. The database file as a whole is not encrypted. No plaintext vault fields are written to it, even temporarily.

Limits. A stolen database still permits offline password guessing; a strong master passphrase and a costly KDF reduce this risk. Existing vault KDF settings must remain stable: changing them or the master password requires re-encrypting the entries, which is outside the initial scope. Forgotten master passwords cannot be recovered. Removing references in Python does not guarantee that every physical copy of a secret has been erased from memory.

3 Authentication sessions and request handling

Users will register with a unique login name and a master passphrase of 15-128 characters (at most 512 UTF-8 bytes). Spaces and special characters are allowed; passwords are not trimmed or silently changed. Validation runs on the server. Failed login responses use one generic message. Planned throttling counts failed attempts by account and by IP, with a starting policy of 5 failures per 5 minutes. A small global limit on simultaneous KDF operations will reduce resource exhaustion.

Session property	Planned behavior
Session ID	A fresh opaque token from 32 cryptographically random bytes. Only the token is sent to the browser.
Server-side state	One process keeps token-to-session records in RAM: user ID, vault key, CSRF token, creation time and last activity. No persistent session store.
Cookie	<code>pm_session; Secure; HttpOnly; SameSite=Strict; Path=/; no Domain; Max-Age=3600.</code> No password or encryption key in the cookie.
Expiry	Reject on the server after 15 minutes idle or 1 hour since login, whichever comes first. Activity updates the idle timer, not the absolute limit.
Fixation prevention	After successful login, discard the anonymous/previous session and create a new token and CSRF token. An old token cannot access the authenticated session.
Logout and cleanup	POST logout deletes server-side state and expires the cookie. Expired state is purged at least once a minute. Restarting the process invalidates all sessions.
Optional MFA	Not in the initial scope. If later required, complete MFA before creating the authenticated session and deriving its vault key.

Flask's default signed client-side session cookie will not store the session record or vault key. Signing a cookie does not encrypt its contents. The planned opaque-token session needs explicit server-side storage and expiry checks. Cookie flags follow the mechanisms described in the lectures and Flask security documentation; `HttpOnly` does not stop an XSS script from reading the page or making authenticated requests.

Authorization and CSRF. Each vault request uses the session's user ID. Read, update and delete queries constrain both entry ID and owner ID. Never trust a submitted `user_id`, a hidden field or a disabled button. All POST forms, including registration, login and logout, use unpredictable session-bound CSRF tokens. Anonymous form sessions contain no key and expire after 15 minutes. GET requests do not change vault data.

Input and output. Limit requests to 64 KiB and apply server-side field limits (title 100, URL 2,048, username 256, saved password 1,024 and notes 2,000 characters). Use parameterized SQL. Render values using Jinja HTML autoescaping in quoted attributes or text nodes; do not use `safe` on user input. Notes are plain text. Do not interpolate user input into JavaScript, use `eval`, or assign it to `innerHTML`. If a DOM update is needed, use `textContent`. Permit only parsed HTTP/HTTPS website URLs with a hostname; initially display them as text and do not fetch them on the server. System error messages come from fixed application text.

TLS and browser protections. For deployment, Nginx will terminate TLS 1.2/1.3 with a trusted certificate and automatic renewal, then proxy to Gunicorn on loopback on the same host. Redirect HTTP to HTTPS and enable HSTS after HTTPS is verified. Only the proxy port is public. Use a trusted development certificate when authentication is introduced locally. The current credential-free scaffold uses loopback HTTP. Send `Cache-Control: no-store`, `Referrer-Policy: no-referrer`, `X-Content-Type-Options: nosniff` and `CSP: same-origin resources, no inline/evaluated scripts, object-src 'none', base-uri 'none', frame-ancestors 'none' and form-action 'self'`. Keep debug mode off and reject unexpected Host headers.

4 Threat model

The mapping uses **OWASP Top 10:2025**, matching lectures. Each row is a scenario for this application and its intended mitigation, not a claim that the control is already implemented.

Threat and impact	Intended mitigation
Input tampering / client-control bypass: changing an entry ID or hidden owner field reads or alters another vault.	Derive user identity from the server session. Check owner on every operation. Validate direct HTTP requests, regardless of browser checks.
HTML injection / content spoofing: a title, note or reflected error creates a fake login prompt.	Escape HTML, allow plain-text notes only, label user content, and use fixed system messages rather than arbitrary URL-supplied errors.
Stored or reflected XSS: a saved entry or reflected login value runs script and exposes unlocked credentials.	Jinja autoescaping at every output, no raw HTML or user-built script strings, and restrictive CSP. Test view and edit pages as well as initial submission.
DOM / JavaScript injection: unsafe DOM insertion or a javascript: website URL executes code.	Avoid <code>innerHTML</code> , <code>document.write</code> , <code>eval</code> and string-to-code APIs. Use <code>textContent</code> ; validate URL schemes on the server.
SQL injection: login or entry fields change a database query.	Bind query parameters with <code>sqlite3</code> placeholders. Never concatenate user input into SQL.
Secret disclosure: intercepted requests, plaintext entries or a stored vault key expose passwords.	HTTPS; separate salted verification and key derivation; server-side AES-GCM; keys only in session RAM; no secret logging.
Authentication/session attacks: password guessing, stolen IDs or session fixation expose a vault.	Argon2id, rate limits, random IDs, new ID after login, cookie flags, server-side expiry and logout invalidation.
CSRF: another website submits an unwanted entry change, deletion or login.	Session-bound CSRF tokens on all POST forms, explicit SameSite setting, and no state changes through GET.
Ciphertext substitution or modification produces incorrect vault data.	AES-GCM tag checks and AAD binding to owner/entry/version. Reject invalid records. This does not prevent deletion or rollback of a valid old database copy.
Misconfiguration exposes debug output, the database or unsafe cross-origin access.	Debug off; DB outside static files; restricted file permissions and Host values; same-origin UI; no permissive credentialed CORS.
A compromised or outdated dependency introduces malicious/vulnerable code.	Keep dependencies small, use official package sources, pin reviewed versions and check advisories before deployment.
Missing security logs hide repeated attacks; verbose logs leak secrets.	Log time, event, user ID where known and outcome for login failures and denied operations. Exclude passwords, keys, tokens and vault content.
Malformed/oversized requests or cryptographic errors exhaust resources or trigger unsafe fallback.	Request/field limits, KDF concurrency limits, bounded sessions, generic errors, failed-tag rejection and transaction rollback.

Because vaults are private, an account's stored payload normally affects that account. A cross-user stored-XSS attack would need another path, such as login CSRF or an access-control flaw. Reflected and DOM paths remain relevant directly.

5 Repository and planned validation

Checkpoint 1 implementation: a starting Flask page (GET /), a simple health endpoint (GET /health), local Host validation and basic security response headers. The README documents scope, routes and tested local startup commands. Authentication, database access, session handling, vault encryption and TLS deployment are planned, not implemented. The local repository is initialized; the student must review, commit and push it using their own repository identity.

Planned route	Purpose
GET, POST /register	Show registration form; create an account
GET, POST /login	Show login form; verify master password and unlock
POST /logout	Revoke session and remove its key
GET /vault	List owned entries
GET, POST /vault/new	Show form; encrypt and create an entry
GET /vault/<int:entry_id>	View one owned entry
GET, POST /vault/<int:entry_id>/edit	Show form; encrypt an updated entry
POST /vault/<int:entry_id>/delete	Delete one owned entry